Contender 7: Autonomous Operator Reflection in Lean

A constructive large-number proposal for the BusyLean repository

Research artifact prepared for the BusyLean project

12 October 2026

Abstract

The BusyLean repository formalizes a Lean edition of the "name the biggest number" contest. Its current Contender 6 applies forest reflection over a finite-type W-language along a universal W-tree that joins every tree definable in that language. This article proposes a next entry, *Contender 7*, by lifting the same construction one type level: from oracle families to *transformers of oracle families*. The primitive reflection operation is packaged as a transformer; arbitrary transformers are run along W-trees; the resulting autonomous lift is itself iterated along the universal tree. The closed Lean definition is

$$\texttt{contender_7} = \mathrm{OpStage}(\mathrm{Ref}, \mathsf{Index})(\mathsf{P5})(42, 42).$$

The supplied Lean proof source establishes the pointwise inequality

$$\texttt{finalFamily}\ n < \texttt{autonomousFamily}\ n\, n,$$

and hence

$$\texttt{contender_6} < \texttt{contender_7}.$$

The proof uses explicit child occurrences and never assumes monotonicity of reflection in its oracle, a property known to fail for the underlying object language. Under the repository's informal calibration $\lambda \approx \psi(\Omega_\omega)$, Contender 6 has scale roughly $f_{\lambda^2}(42)$, while the operator-level autonomous closure proposed here has the heuristic scale $f_{\lambda^\lambda}(42)$. The strict inequality is a formal Lean theorem statement with proof source; the ordinal calibration is deliberately presented only as a heuristic. The bundle also contains source audits, smoke tests, a clean-build script, a unified patch, and this compiled article. Because the execution environment used to prepare the bundle lacks Lean 4.34.0, final kernel compilation and timing remain a required integration check rather than a claimed result.

Contents

1 Problem and repository baseline **3**
1.1 Why a mere arithmetic wrapper is inadequate . . . 3

2 Contender 6 in functional form **3**
2.1 Families and ordinary reflection . . . 3
2.2 Forest reflection . . . 4
2.3 The universal index . . . 4

3 From functions to operators **4**
3.1 Transformers . . . 4
3.2 Running a transformer along a tree . . . 5
3.3 Autonomous lifting . . . 5

4 Iterating the lift itself **5**
4.1 Splicing child transformers . . . 6
4.2 The outer recursion . . . 6

5 The proposed number **6**
5.1 Computability in principle . . . 7

6 Direct comparison without monotonicity **7**
6.1 Progressive transformers . . . 7
6.2 Child dominance . . . 7
6.3 A transformer lies below its autonomous lift . . . 8
6.4 Contender 6 is a visible outer leaf . . . 8

7 A stronger closure theorem **9**

8 Informal size analysis **9**
8.1 Repository calibration . . . 10
8.2 One type-level lift adds an exponent . . . 10
8.3 Closing over the universal outer tree . . . 10

9 Why not claim the System F jump yet? **11**

10 Lean engineering and dependency discipline **11**
10.1 Definition/proof separation . . . 11
10.2 Excluded constructs . . . 11
10.3 Avoiding accidental normalization . . . 12
10.4 Expected axiom profile . . . 12

11 Validation protocol and current status **12**
11.1 Checks completed in this bundle . . . 12
11.2 Decisive check still required . . . 12
11.3 Likely integration failure modes . . . 13

12 Integration instructions **13**

13 Further research questions **13**

14 Conclusion **14**

A Definition source **14**
A.1 `MetaReflection.lean` . . . 14
A.2 `Contender7.lean` . . . 15

B Proof source **16**
B.1 `MetaReflectionFacts.lean` . . . 16
B.2 `Comparison7.lean` . . . 18

C Tests **19**
C.1 Smoke test . . . 19
C.2 Axiom audit . . . 20

1 Problem and repository baseline

BusyLean changes the usual large-number game in two decisive ways. A contender must denote a closed, computable natural number, and superiority over the previous contender must be justified by a proof checked by Lean's kernel. The rules exclude nonconstructive or implementation-only escape hatches such as `noncomputable`, `partial`, `unsafe`, opaque stand-ins, `implemented_by`, external primitives, `native_decide`, custom axioms, and appeals to uncomputable functions such as Busy Beaver. They also impose separate checking budgets for definitions and proofs.

The repository revision used by this artifact is

`86b360648163f0249eab79539438b6a83dfe8215`.

At that revision the incumbent is

```
def contender_6 : Nat := stage p5Family index 42
```

and the principal comparison theorem is

```
theorem contender_5_lt_contender_6 : contender_5 < contender_6
```

The source is organized so that the definition layer can be checked without normalizing the enormous number. This proposal preserves that architecture: definitions live in `MetaReflection.lean` and `Contender7.lean`; proofs live in `MetaReflectionFacts.lean` and `Comparison7.lean`.

1.1 Why a mere arithmetic wrapper is inadequate

There are trivial ways to obtain a larger number, for example $\texttt{contender_6} + 1$ or $2^{\texttt{contender_6}}$. Such definitions are computable and easy to compare, but they violate the scientific intent of the contest's "no laziness" rule: they exploit the previous contender as data and add only a superficial arithmetic shell. The new definition must instead strengthen the *generating mechanism* and should not mention the constant `contender_6` at all.

Contender 7 meets this stronger criterion. Its definition imports the W-language, universal tree, and ordinary-reflection machinery behind Contender 6, but not the previous named value. It introduces a new recursion at the operator level and proves the old family is embedded strictly below it.

2 Contender 6 in functional form

The design of Contender 7 is easiest to understand by restating the incumbent as a functional construction.

2.1 Families and ordinary reflection

Let

$$\mathcal{F} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}$$

be the type of countable families of unary numerical functions. In Lean this is the abbreviation

```
abbrev Family := Nat → Nat → Nat
```

The first coordinate selects an oracle member; the second is the numerical input. The W-language interpreter is evaluated in an environment whose oldest variable denotes such a family. The operation $\mathrm{Ref}(fs, n)$ strictly exceeds both:

1. every visible oracle value $fs(i, n)$ with $i \leq n$, and

2. every enumerated program of depth at most n, evaluated relative to fs.

The implementation is a finite maximum plus a positive offset, so it is constructive and total.

2.2 Forest reflection

The type `WTree Nat` has a leaf constructor and countably branching nodes:

$$a ::= \mathsf{leaf} \mid \mathsf{node}(f), \qquad f : \mathbb{N} \to \mathsf{WTree}(\mathbb{N}).$$

Ordinary forest reflection is

$$\begin{aligned} \mathsf{stage}(s, \mathsf{leaf})(n) &= \mathrm{Ref}(s, n), \\ \mathsf{stage}(s, \mathsf{node}(f))(n) &= \mathrm{Ref}\big((i, m) \mapsto \mathsf{stage}(s, f(i))(m),\ n\big). \end{aligned}$$

At a node, the child stages are retained as a family rather than collapsed pointwise. A program may therefore compute which child to call. This is crucial: a simple pointwise maximum would erase the dynamic indexing power of the oracle language.

2.3 The universal index

For a program t, `defTree t` interprets it and casts the result to a `WTree Nat`, using a leaf on type mismatch. The tree `treesOfDepth k` joins the trees denoted by every program enumerated at depth at most k. Finally,

```
def index : WTree Nat := .node treesOfDepth
```

joins those finite-depth joins for every k. Thus `index` does not encode a hand-written ordinal notation system; it ranges over every W-tree that the object language itself defines.

Writing Index for this tree and $s = \mathsf{P5}$ for the P5 seed family, Contender 6 is

$$C_6 = \mathsf{stage}(s, \mathsf{Index})(42).$$

Equivalently, after packaging reflection as a family transformer, it will be one autonomous lift of the primitive transformer. This reformulation reveals the next natural closure operation.

3 From functions to operators

3.1 Transformers

Definition 3.1 (Transformer)**.** A transformer is a function from oracle families to oracle families:

$$\mathcal{T} = \mathcal{F} \to \mathcal{F}.$$

In Lean:

```
abbrev Transformer := Family → Family
```

The ordinary reflection step is packaged as the transformer

$$R(fs)(o, n) = \mathrm{Ref}(fs, n),$$

where the output coordinate o is intentionally ignored.

```
def reflectionTransformer : Transformer :=
  fun fs _ n => reflect fs n
```

The redundant coordinate is not cosmetic. Once transformers are combined, that coordinate becomes the routing channel through which a higher-level operator selects a child transformer.

3.2 Running a transformer along a tree

Definition 3.2 (Transformer run)**.** For $T \in \mathcal{T}$, seed $s \in \mathcal{F}$, and W-tree a, define $\mathrm{Run}(T, s, a) \in \mathcal{F}$ recursively by

$$\mathrm{Run}(T, s, \mathsf{leaf}) = T(s),$$
$$\mathrm{Run}(T, s, \mathsf{node}(f)) = T\big((i, n) \mapsto \mathrm{Run}(T, s, f(i))(i, n)\big).$$

The aligned coordinate i in the recursive clause is important. Coordinate i of the family supplied to T contains child i, evaluated at child output coordinate i. Thus a transformer can inspect or route through the entire countable forest.

The Lean definition is structural:

```
def runTransformer (op : Transformer) (seed : Family) :
    WTree Nat → Family
  | .leaf => op seed
  | .node f =>
      op (fun i n => runTransformer op seed (f i) i n)
```

No general recursion, termination oracle, or postulated well-founded relation is required; recursion is directly on the inductive W-tree.

3.3 Autonomous lifting

Definition 3.3 (Autonomous lift)**.** The lift of a transformer runs it along the universal index:

$$\mathrm{Lift}(T)(s) = \mathrm{Run}(T, s, \mathsf{Index}).$$

```
def liftTransformer (op : Transformer) : Transformer :=
  fun seed => runTransformer op seed index
```

For the primitive reflection transformer, this recovers the incumbent forest reflection exactly.

Theorem 3.4 (Compatibility with Contender 6)**.** *For every seed family s, output coordinate o, and input n,*

$$\mathrm{Lift}(R)(s)(o, n) = \mathsf{stage}(s, \mathsf{Index})(n).$$

The supplied Lean theorem is

```
theorem liftReflection_eq_stage (seed : Family) (o n : Nat) :
    liftTransformer reflectionTransformer seed o n =
      stage seed index n
```

It follows by structural induction on the W-tree, not by extensional or ordinal reasoning.

4 Iterating the lift itself

A single application of Lift turns ordinary reflection into the mechanism behind Contender 6. Contender 7 makes Lift itself the operation that is iterated.

4.1 Splicing child transformers

Given a countable family $(T_i)_{i\in\mathbb{N}}$ of transformers, define their splice by

$$\mathsf{splice}(T_0, T_1, \ldots)(s)(i, n) = T_i(s)(0, n).$$

Output coordinate i chooses the i-th transformer. Coordinate zero is the canonical observation point inside the selected child.

```
def spliceTransformers (ops : Nat → Transformer) : Transformer :=
  fun seed i n => ops i seed 0 n
```

4.2 The outer recursion

Definition 4.1 (Operator stage)**.** For a base transformer B, define a tree-indexed transformer $\mathrm{OpStage}(B, a)$ by

$$\mathrm{OpStage}(B, \mathsf{leaf}) = \mathrm{Lift}(B),$$
$$\mathrm{OpStage}(B, \mathsf{node}(f)) = \mathrm{Lift}\big(\mathsf{splice}(i \mapsto \mathrm{OpStage}(B, f(i)))\big).$$

In Lean:

```
def operatorStage (base : Transformer) : WTree Nat → Transformer
  | .leaf => liftTransformer base
  | .node f =>
      liftTransformer
        (spliceTransformers (fun i => operatorStage base (f i)))
```

This is the central new definition. There are now two autonomous W-tree layers:

1. the *inner* universal index inside every application of Lift, and

2. the *outer* tree controlling how those lifted transformers are spliced and lifted again.

Choosing the universal index for the outer tree closes the hierarchy under all operator stages generated by trees definable in the same W-language.

5 The proposed number

Construction 5.1 (Contender 7)**.** Define

$$A = \mathrm{OpStage}(R, \mathsf{Index}),$$
$$F = A(\mathsf{P5}),$$
$$C_7 = F(42, 42).$$

The complete definition layer is only:

```
def autonomousTransformer : Transformer :=
  operatorStage reflectionTransformer index

def autonomousFamily : Family :=
  autonomousTransformer p5Family

def contender_7 : Nat := autonomousFamily 42 42
```

The diagonal coordinate $(42, 42)$ is deliberate. The first 42 exposes the outer transformer's output channel, while the second 42 is the ordinary numerical input and the visibility bound used by progressivity. No occurrence of the constant `contender_6` appears in these definitions.

5.1 Computability in principle

Every component is executable:

- program enumerations at a fixed depth are finite lists;
- the interpreter of the finite-type W-language is total;
- `reflect` computes finite maxima and arithmetic;
- `runTransformer` and `operatorStage` recurse structurally on inductive W-trees;
- `index` is a closed W-tree whose branches are generated by total functions; and
- the seed `p5Family` is already a total function in the repository.

Consequently `contender_7` is a closed `Nat`. "Computable in principle" does not mean feasible: evaluating it would require an inconceivable amount of computation and must not be attempted. Lean checks its definition and proofs symbolically, without normalizing the numeral.

6 Direct comparison without monotonicity

The key technical risk is monotonicity. A program in the object language can inspect an oracle value and branch on it; increasing an oracle pointwise can therefore decrease a program's output. Neither `reflect` nor the lifted operators should be treated as monotone in their oracle argument. The proof instead records explicit occurrences.

6.1 Progressive transformers

Definition 6.1 (Progressivity)**.** A transformer T is progressive when

$$i \le n \quad \Longrightarrow \quad s(i,n) < T(s)(o,n)$$

for every seed s, every input n, and every output coordinate o.

The Lean predicate is

```
def Progressive (op : Transformer) : Prop :=
  ∀ (seed : Family) (i o n : Nat),
    i ≤ n → seed i n < op seed o n
```

Ordinary reflection is progressive by the repository theorem `oracle_lt_reflect`.

6.2 Child dominance

At a node, a visible aligned child is literally one coordinate of the family supplied to the parent transformer. Progressivity therefore gives the strict inequality directly.

Lemma 6.2 (Run child)**.** *If T is progressive and $i \le n$, then*

$$\mathrm{Run}(T, s, f(i))(i,n) < \mathrm{Run}(T, s, \mathsf{node}(f))(o,n).$$

The corresponding Lean theorem is `runTransformer_child_lt`. No comparison between two different oracle families is made.

6.3 A transformer lies below its autonomous lift

The universal index has the form

$$\mathsf{Index} = \mathsf{node}(k \mapsto \mathsf{treesOfDepth}(k)).$$

At depth zero, the term enumeration is empty. Every defaulted child of `treesOfDepth 0` is therefore a leaf. For any visible coordinate $i \le n$, there is an explicit two-edge path

$$\mathsf{leaf} \longrightarrow \mathsf{treesOfDepth}(0) \longrightarrow \mathsf{Index}.$$

Applying the run-child lemma on each edge yields:

Theorem 6.3 (Lift dominance)**.** *If T is progressive and $i \le n$, then*

$$T(s)(i,n) < \mathrm{Lift}(T)(s)(o,n).$$

This is formalized as `transformer_lt_lift_at`. It immediately gives that autonomous lifting preserves progressivity. Splicing also preserves progressivity when every selected child transformer is progressive. A structural induction then proves:

Theorem 6.4 (Progressivity of operator stages)**.** *If the base transformer B is progressive, then* $\mathrm{OpStage}(B,a)$ *is progressive for every W-tree a.*

A second direct child lemma gives

$$\mathrm{OpStage}(B, f(i))(s)(0,n) < \mathrm{OpStage}(B, \mathsf{node}(f))(s)(o,n) \qquad (i \le n).$$

6.4 Contender 6 is a visible outer leaf

At an outer leaf,

$$\mathrm{OpStage}(R, \mathsf{leaf}) = \mathrm{Lift}(R).$$

By the compatibility theorem in section 3, this is exactly the Contender 6 family. The same two explicit outer edges

$$\mathsf{leaf} \longrightarrow \mathsf{treesOfDepth}(0) \longrightarrow \mathsf{Index}$$

strictly dominate that leaf transformer. Therefore:

Theorem 6.5 (Pointwise formal margin)**.** *For every $n \in \mathbb{N}$,*

$$\mathit{finalFamily}\ n < \mathit{autonomousFamily}\ n\ n.$$

The supplied proof is:

```
theorem finalFamily_lt_autonomousFamily (n : Nat) :
    finalFamily n < autonomousFamily n n := by
  have h0 :
      operatorStage reflectionTransformer .leaf p5Family 0 n <
        operatorStage reflectionTransformer
          (treesOfDepth 0) p5Family 0 n := by
    have h := operatorStage_child_lt reflectionTransformer
      reflectionTransformer_progressive p5Family
      (fun i => defTree
        (nthD (BigNumber.allTerms 0) i .zero))
      (i := 0) (o := 0) (n := n) (Nat.zero_le n)
    rw [depthZeroTreeLeaf] at h
    exact h
```

```
  have h1 :
      operatorStage reflectionTransformer
          (treesOfDepth 0) p5Family 0 n <
        operatorStage reflectionTransformer
          index p5Family n n :=
    operatorStage_child_lt reflectionTransformer
      reflectionTransformer_progressive p5Family treesOfDepth
      (i := 0) (o := n) (n := n) (Nat.zero_le n)
  have h :
      liftTransformer reflectionTransformer p5Family 0 n <
        operatorStage reflectionTransformer index p5Family n n :=
    Nat.lt_trans h0 h1
  change stage p5Family index n <
    operatorStage reflectionTransformer index p5Family n n
  rw [← liftReflection_eq_stage p5Family 0 n]
  exact h
```

Specializing to 42 proves the contest comparison:

```
theorem contender_6_lt_contender_7 :
    contender_6 < contender_7 :=
  finalFamily_lt_autonomousFamily 42
```

By transitivity, the bundle also derives strict inequalities from the exact Contender 5 port and from proposal P5's number.

7 A stronger closure theorem

The definition is designed to exceed more than one named constant. Let t be any enumerated object-language program denoting a W-tree. If t appears at finite enumeration depth k, then `defTree t` is a child of `treesOfDepth k`, and that tree is a child of `index`. Reusing direct child dominance at the operator level yields the theorem

```
theorem defined_operator_eventually_lt {k : Nat} {t : Term}
    (ht : t ∈ BigNumber.allTerms k) :
    ∃ N, ∀ n, N ≤ n →
      operatorStage reflectionTransformer (defTree t)
          p5Family 0 n <
        autonomousFamily n n
```

Thus every operator-stage whose outer schedule is denoted by a fixed program of the W-language is eventually dominated by the autonomous family. This is the meaningful structural margin: Contender 7 closes not only over the old number, but over a whole class of independently generated operator hierarchies.

The theorem is still deliberately modest. It does not claim that every total Lean transformer is represented, nor does it quantify over arbitrary semantic well-orderings. Its scope is exactly the effective W-language already audited by BusyLean.

8 Informal size analysis

This section is heuristic. None of its ordinal identifications is used in the Lean comparison proof.

8.1 Repository calibration

Let λ denote the proof-theoretic scale of the finite-type W-language. The repository informally calibrates it near

$$\lambda \approx \psi(\Omega_\omega).$$

One reflection step relative to an oracle is treated as adding about λ to the fast-growing index. Iteration along a W-tree of height α then has the rough effect of right multiplication by α. Because the universal index ranges over the language's own definable trees, its effective height is heuristically λ. On that account:

$$C_6 \quad \text{has scale roughly} \quad f_{\lambda\cdot\lambda}(42) = f_{\lambda^2}(42).$$

8.2 One type-level lift adds an exponent

Suppose, schematically, that a progressive transformer has increment scale λ^e. Running it along the universal index composes that increment through height λ, suggesting

$$\lambda^e \cdot \lambda = \lambda^{e+1}.$$

The primitive reflection transformer starts at exponent one. Its autonomous lift—the mechanism of Contender 6—is therefore at exponent two.

For a finite outer tree of rank h, `operatorStage` applies one further universal lift at each outer level. The finite skeleton is

outer height	heuristic increment scale
0	λ^2
1	λ^3
2	λ^4
$\vdots$	$\vdots$
h	λ^{h+2}.

The included script `scripts/ordinal_skeleton.py` regression-tests this finite bookkeeping model.

8.3 Closing over the universal outer tree

The outer tree is again Index, heuristically cofinal through ranks below λ. Taking the supremum of the finite and transfinite outer stages therefore suggests

$$\sup_{\alpha<\lambda} \lambda^{\alpha+2} = \lambda^\lambda.$$

Accordingly, the proposed calibration is

$$C_7 \quad \text{roughly} \quad f_{\lambda^\lambda}(42), \qquad \lambda \approx \psi(\Omega_\omega).$$

This is a substantial mechanism-level jump over $f_{\lambda^2}(42)$, while remaining within the same constructive object language.

Remark 8.1 (Limits of the calibration). The equation "Contender 7 is $f_{\lambda^\lambda}(42)$" is not a theorem of this work. A rigorous calibration would require an ordinal notation system, a semantics relating transformer runs to that system, upper and lower bounds, and a proof that the universal tree enumeration is cofinal in the required sense. The present Lean development proves strict numerical inequalities and structural domination theorems, not ordinal equivalences.

9 Why not claim the System F jump yet?

The existing BusyLean article identifies impredicative polymorphism as a promising qualitative next step. A fully satisfactory System F contender would require considerably more infrastructure:

1. syntax and decidable typing for a suitable presentation of System F;
2. substitution, reduction, and a deterministic normalizer;
3. a formal strong-normalization proof, most naturally through reducibility candidates in Lean's impredicative `Prop`;
4. a verified decoder for Church numerals;
5. a finite bounded syntax enumeration; and
6. an oracle extension whose normalization proof remains constructive.

Such a development could plausibly dwarf the predicative W-language, but an uncompiled sketch would not satisfy the contest. The present proposal instead makes a smaller foundational change that can be expressed entirely using the already audited W-language and structural recursion. It is intended as a credible, reviewable Contender 7 rather than a promise of a stronger but unfinished system.

10 Lean engineering and dependency discipline

10.1 Definition/proof separation

The new files follow the incumbent's time-budget architecture:

File	Role
`MetaReflection.lean`	Definition-only operator machinery.
`Contender7.lean`	Closed family and number definitions.
`MetaReflectionFacts.lean`	Progressivity, compatibility, and direct child inequalities.
`Comparison7.lean`	Named-contender comparisons and autonomous closure theorem.
`MetaReflectionSmoke.lean`	Tiny executable recursion checks.
`Contender7Audit.lean`	Axiom reports and theorem signature checks.

10.2 Excluded constructs

A source audit checks the Lean code, after removing comments, for the forbidden patterns

`noncomputable`, `partial`, `unsafe`, `opaque`, `implemented_by`, `extern`, `native_decide`, `sorry`, `admit`, and custom `axiom` declarations.

It also checks that the definition layer does not mention `contender_6`; that expected declarations exist; and that the aggregate `BusyLean.lean` imports all new modules. The Lean-side audit goes further: it reconstructs the transitive declaration closure from the kernel environment and is set to reject dependencies on the named Contender 6 and Contender 5 constants or selectors. That closure check becomes authoritative only when the supplied test is run by Lean.

10.3 Avoiding accidental normalization

The comparison proof first proves a general pointwise inequality in a symbolic variable n, and only then specializes to 42. The proof never unfolds the astronomical arithmetic computation. Rewriting is directed through `liftReflection_eq_stage`, so elaboration needs only definitional unfolding of the construction, not evaluation of the number.

10.4 Expected axiom profile

The mathematical proof uses direct natural-number inequalities, structural induction, function extensionality in the compatibility lemma, and existing BusyLean results. On Lean 4.34.0, the repository permits `propext` and `Quot.sound`; the audit script rejects any reported axiom outside that set. The exact reported set must be established by the final kernel run.

11 Validation protocol and current status

11.1 Checks completed in this bundle

The preparation environment performed the following checks:

1. source-level audit of all new Lean and test files;
2. verification that no forbidden constructive-rule token occurs in code;
3. verification that the definition layer is independent of the named constant `contender_6`;
4. verification that the expected definitions, theorem names, comparison chain, and imports are present;
5. execution of the finite ordinal-exponent skeleton script;
6. Python bytecode compilation of the validation scripts;
7. two-pass LaTeX compilation of this article;
8. PDF metadata inspection, text extraction, and page rendering; and
9. visual inspection of the rendered pages for clipping and glyph errors.

11.2 Decisive check still required

Validation status 11.1. The current execution environment contains no Lean or Lake executable and has no network access from which to install the pinned Lean 4.34.0 binary. I therefore do *not* claim that the supplied Lean source has been accepted by Lean's kernel or that it satisfies the 15-second/60-second budgets on this machine. The source has been manually and statically reviewed, but kernel checking is authoritative.

The bundle supplies `scripts/check_contender7.py`. After applying the overlay to the recorded BusyLean revision, run

```
python3 scripts/check_contender7.py
```

The script deletes build directories, builds `BusyLean.Contender7` under the definition budget, builds the proof layer under the proof budget, executes the smoke test, runs the axiom audit, and rejects any axiom outside {`propext`, `Quot.sound`}.

11.3 Likely integration failure modes

The most plausible issues are engineering rather than mathematical:

- Lean may require a small elaboration adjustment in a higher-order structural recursion equation;
- a rewrite may need an explicit unfolding directive to prevent the elaborator from choosing an undesirable normal form;
- the toolchain may attribute `propext` or `Quot.sound` through core lemmas, as already happens elsewhere in the repository; or
- clean-build time may exceed the contest budget despite short source.

None should be concealed. If the kernel rejects a term, the theorem is not yet established, regardless of the paper proof. The validation report in the bundle records this distinction.

12 Integration instructions

The archive contains a `repo_overlay/` directory mirroring the repository root and a unified patch in `patches/contender7.patch`. From a checkout at the recorded commit:

```
git apply patches/contender7.patch
python3 scripts/check_contender7.py
```

Alternatively copy the overlay into the checkout, preserving paths. The aggregate file `BusyLean.lean` gains four imports:

```
import BusyLean.MetaReflection
import BusyLean.MetaReflectionFacts
import BusyLean.Contender7
import BusyLean.Comparison7
```

No existing definition is modified. The proposal is therefore easy to remove or review in isolation.

13 Further research questions

1. **Kernel stabilization.** Compile on Lean 4.34.0, minimize any elaboration fixes, and preserve the definition/proof timing split.
2. **Richer dependency-closure reporting.** After the bundled rejection test passes, list and classify all inherited constants under the old-language and proposal namespaces, as the Contender 6 audit does.
3. **Tighter structural theorem.** Formalize domination of every finite iteration of `liftTransformer`, perhaps using canonical spine trees and an explicit embedding into `index`.
4. **Representation theorem.** Characterize which progressive transformers can be represented by W-language programs or by `operatorStage` schedules.
5. **Ordinal lower bound.** Define an ordinal notation semantics for the operator hierarchy and prove a lower bound approaching λ^λ, rather than relying on the present calibration heuristic.

6. **Ordinal upper bound.** Show that the recursion does not silently exceed the proposed calibration because of higher-type oracle routing. A matching upper bound would make the ordinal claim meaningful.

7. **Higher transformer levels.** Iterate the construction at types $\mathcal{F} \to \mathcal{F}$, $(\mathcal{F} \to \mathcal{F}) \to (\mathcal{F} \to \mathcal{F})$, and beyond, while retaining a direct, non-monotone comparison principle.

8. **Diagonal autonomous tower.** Encode all finite transformer levels in one inductive universe of codes and take a verified diagonal across levels. This may suggest scales analogous to $\lambda \uparrow\uparrow \lambda$, but requires careful typing.

9. **Impredicative successor.** Complete a constructive System F normalization development, then compare a bounded-depth System F maximum against Contender 7 without invoking classical choice.

10. **Calculus of Constructions.** Investigate a Lean-hosted, independently normalized dependent type theory in the direction of Loader-style numbers, with a faithful and auditable syntax bound.

11. **Complexity of checking.** Measure elaboration and kernel costs separately. A definition may be mathematically compact yet expensive for Lean because of generated recursors or closure size.

12. **Robustness under toolchain upgrades.** Test the axiom profile and generated code on subsequent Lean releases, documenting changes due solely to core implementation details.

14 Conclusion

Contender 7 replaces a one-level autonomous reflection hierarchy by an autonomous hierarchy of reflection *operators*. The proposal remains a closed, structurally recursive Lean natural number and does not mention the previous contender constant. Its formal comparison strategy is deliberately conservative: define progressivity, follow explicit child occurrences, identify the outer leaf with the Contender 6 family, and compose two strict inequalities. The same argument proves eventual domination of every operator-stage generated by a fixed W-language-definable tree.

The construction is therefore substantially more than an arithmetic wrapper. Under the repository's own proof-theoretic heuristic it moves from λ^2 to λ^λ, with $\lambda \approx \psi(\Omega_\omega)$. That estimate is not passed off as a formal theorem. The formal deliverable is the Lean source for `contender_7`, its pointwise comparison theorem, and the associated closure lemmas. The remaining gate is unambiguous: apply the patch to the recorded repository revision and pass the supplied Lean 4.34.0 clean-build, timing, smoke, and axiom checks.

A Definition source

A.1 `MetaReflection.lean`

```
import BusyLean.Contender6

/-!
# Reflection lifted to transformers

Contender 6 applies forest reflection once, along `index`, to a seed family.
This file lifts that construction by one type level.

A `Transformer` maps oracle families to oracle families.  The primitive
transformer performs one ordinary reflection step.  `runTransformer` iterates
an arbitrary transformer along a `WTree Nat`; `liftTransformer` runs it along
the universal tree `index`.  Finally, `operatorStage` iterates this lift itself
```

```
along another W-tree.

All definitions are structural and executable.  There is no use of
`noncomputable`, `partial`, `unsafe`, `opaque`, `implemented_by`, or `extern`.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- An operation taking one countable oracle family to another. -/
abbrev Transformer := Family → Family

/-- One ordinary reflection step, packaged as a transformer.
The output-family index is deliberately ignored. -/
def reflectionTransformer : Transformer :=
  fun fs _ n => reflect fs n

/-- Assemble a countable family of transformers into one transformer.
Output coordinate `i` selects transformer `i`; the selected transformer's
canonical output coordinate is `0`. -/
def spliceTransformers (ops : Nat → Transformer) : Transformer :=
  fun seed i n => ops i seed 0 n

/-- Iterate a transformer along a well-founded tree.

At a node, coordinate `i` of the input family supplied to `op` is the result of
running the same transformer down child `i`, evaluated at coordinate `i`.
Thus an operator may select a child dynamically through its family argument. -/
def runTransformer (op : Transformer) (seed : Family) :
    WTree Nat → Family
  | .leaf => op seed
  | .node f =>
      op (fun i n => runTransformer op seed (f i) i n)

/-- The autonomous lift of a transformer: iterate it along the same universal
index that Contender 6 uses. -/
def liftTransformer (op : Transformer) : Transformer :=
  fun seed => runTransformer op seed index

/-- Iterate the lifting operation itself along a W-tree.

* At a leaf, lift the base transformer once.
* At a node, splice the recursively generated child transformers, then lift
  the splice along `index`.

Running this construction on `index` is the second, operator-level autonomous
closure used by Contender 7. -/
def operatorStage (base : Transformer) : WTree Nat → Transformer
  | .leaf => liftTransformer base
  | .node f =>
      liftTransformer
        (spliceTransformers (fun i => operatorStage base (f i)))

end BusyLean
```

A.2 `Contender7.lean`

```
import BusyLean.MetaReflection

/-!
# Contender 7: autonomous operator reflection

Contender 6 performs one autonomous reflection pass

```
```
```

```
stage p5Family index
```

where `index` joins every `W Nat` tree defined by the W-language. Contender 7
lifts this operation to the type of family transformers and then runs the lift
itself along the same universal index.

Informally, if the W-language has strength `lambda ~= psi(Omega_omega)`, one
ordinary reflection step contributes about `lambda`, Contender 6 contributes
about `lambda^2`, and this second autonomous closure has the exponentiation-like
calibration `lambda^lambda`. This calibration is heuristic; the strict
comparison with Contender 6 is formalized in `BusyLean.Comparison7`.

Never evaluate `contender_7`.
-/

namespace BusyLean

open BusyLean.W

/-- The universal operator-level reflection transformer. -/
def autonomousTransformer : Transformer :=
  operatorStage reflectionTransformer index

/-- The resulting two-coordinate family. -/
def autonomousFamily : Family :=
  autonomousTransformer p5Family

/-- **Contender 7.** The diagonal coordinate is used deliberately. -/
def contender_7 : Nat := autonomousFamily 42 42

end BusyLean
```

# B Proof source

## B.1 MetaReflectionFacts.lean

```
import BusyLean.MetaReflection
import BusyLean.ReflectionFacts

/-!
Direct inequalities for transformer reflection

As in `BusyLean.ReflectionFacts`, no monotonicity of reflection in its oracle is
assumed. It is false in general because object-language programs may inspect
and branch on oracle values. Every strict inequality below follows an explicit
occurrence: a visible seed coordinate or a visible child of a W-tree.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- A transformer is progressive when every visible seed value is strictly
below every output coordinate at the same numerical input. -/
def Progressive (op : Transformer) : Prop :=
  ∀ (seed : Family) (i o n : Nat), i ≤ n → seed i n < op seed o n

/-- Ordinary reflection is progressive. -/
theorem reflectionTransformer_progressive :
    Progressive reflectionTransformer := by
  intro seed i o n hi
  exact oracle_lt_reflect seed hi

/-- Splicing preserves progressivity when every selected transformer is
```
```

```
progressive. -/
theorem spliceTransformers_progressive {ops : Nat → Transformer}
    (hops : ∀ i, Progressive (ops i)) :
    Progressive (spliceTransformers ops) := by
  intro seed i o n hi
  exact (hops o) seed i 0 n hi

/-- At a transformer-run node, every visible aligned child is strictly below
its parent. -/
theorem runTransformer_child_lt {op : Transformer}
    (hop : Progressive op) (seed : Family) (f : Nat → WTree Nat)
    {i o n : Nat} (hi : i ≤ n) :
    runTransformer op seed (f i) i n <
      runTransformer op seed (.node f) o n := by
  change runTransformer op seed (f i) i n <
    op (fun j m => runTransformer op seed (f j) j m) o n
  exact hop (fun j m => runTransformer op seed (f j) j m) i o n hi

/-- The primitive transformer-run is exactly the old `stage`, independently
of the output coordinate. -/
theorem runTransformer_reflection_eq_stage (seed : Family) :
    ∀ (a : WTree Nat) (o n : Nat),
      runTransformer reflectionTransformer seed a o n = stage seed a n
  | .leaf, _, _ => rfl
  | .node f, o, n => by
      change reflect
          (fun i m => runTransformer reflectionTransformer seed (f i) i m) n =
        reflect (fun i m => stage seed (f i) m) n
      apply congrArg (fun fs : Family => reflect fs n)
      funext i m
      exact runTransformer_reflection_eq_stage seed (f i) i m

/-- Consequently, lifting ordinary reflection is exactly one Contender-6
forest-reflection pass. -/
theorem liftReflection_eq_stage (seed : Family) (o n : Nat) :
    liftTransformer reflectionTransformer seed o n = stage seed index n :=
  runTransformer_reflection_eq_stage seed index o n

/-- Every child of the depth-zero join is a leaf. -/
theorem depthZeroTreeLeaf (i : Nat) :
    defTree (nthD (BigNumber.allTerms 0) i .zero) = .leaf := rfl

/-- A progressive transformer is strictly below its autonomous lift at every
visible output coordinate.  The proof follows the explicit path

`leaf -> treesOfDepth 0 -> index`.
-/
theorem transformer_lt_lift_at {op : Transformer}
    (hop : Progressive op) (seed : Family) {i o n : Nat} (hi : i ≤ n) :
    op seed i n < liftTransformer op seed o n := by
  have h1 :
      runTransformer op seed
          (defTree (nthD (BigNumber.allTerms 0) i .zero)) i n <
        runTransformer op seed (treesOfDepth 0) 0 n :=
    runTransformer_child_lt hop seed
      (fun j => defTree (nthD (BigNumber.allTerms 0) j .zero)) hi
  have h2 :
      runTransformer op seed (treesOfDepth 0) 0 n <
        runTransformer op seed index o n :=
    runTransformer_child_lt hop seed treesOfDepth (Nat.zero_le n)
  rw [depthZeroTreeLeaf] at h1
  exact Nat.lt_trans h1 h2

/-- Autonomous lifting preserves progressivity. -/
theorem liftTransformer_progressive {op : Transformer}
    (hop : Progressive op) : Progressive (liftTransformer op) := by
  intro seed i o n hi
  exact Nat.lt_trans (hop seed i i n hi)
    (transformer_lt_lift_at hop seed hi)
```

```
/-- Every operator generated by `operatorStage` is progressive. -/
theorem operatorStage_progressive (base : Transformer)
    (hbase : Progressive base) :
    (a : WTree Nat) → Progressive (operatorStage base a)
  | .leaf => liftTransformer_progressive hbase
  | .node f =>
      liftTransformer_progressive
        (spliceTransformers_progressive
          (fun i => operatorStage_progressive base hbase (f i)))

/-- Every visible child operator is strictly below its operator-stage parent.
The child is used at its canonical output coordinate `0`; coordinate `i` of the
splice names child `i`. -/
theorem operatorStage_child_lt (base : Transformer)
    (hbase : Progressive base) (seed : Family) (f : Nat → WTree Nat)
    {i o n : Nat} (hi : i ≤ n) :
    operatorStage base (f i) seed 0 n <
      operatorStage base (.node f) seed o n := by
  change spliceTransformers (fun j => operatorStage base (f j)) seed i n <
    liftTransformer
      (spliceTransformers (fun j => operatorStage base (f j))) seed o n
  exact transformer_lt_lift_at
    (spliceTransformers_progressive
      (fun j => operatorStage_progressive base hbase (f j))) seed hi

end BusyLean
```

B.2 Comparison7.lean

```
import BusyLean.Contender7
import BusyLean.MetaReflectionFacts
import BusyLean.Comparison

/-!
# Contender 7: comparison theorems

The required theorem is `contender_6_lt_contender_7`.  The proof does not use
monotonicity.  It exposes Contender 6 as the leaf operator and then follows two
visible child edges through the outer universal index.
-/

namespace BusyLean

open BigNumber (Term)
open BusyLean.W

/-- At every input, the autonomous operator family strictly exceeds the entire
Contender-6 family. -/
theorem finalFamily_lt_autonomousFamily (n : Nat) :
    finalFamily n < autonomousFamily n n := by
  have h0 :
      operatorStage reflectionTransformer .leaf p5Family 0 n <
        operatorStage reflectionTransformer (treesOfDepth 0) p5Family 0 n := by
    have h := operatorStage_child_lt reflectionTransformer
      reflectionTransformer_progressive p5Family
      (fun i => defTree (nthD (BigNumber.allTerms 0) i .zero))
      (i := 0) (o := 0) (n := n) (Nat.zero_le n)
    rw [depthZeroTreeLeaf] at h
    exact h
  have h1 :
      operatorStage reflectionTransformer (treesOfDepth 0) p5Family 0 n <
        operatorStage reflectionTransformer index p5Family n n :=
    operatorStage_child_lt reflectionTransformer
      reflectionTransformer_progressive p5Family treesOfDepth
      (i := 0) (o := n) (n := n) (Nat.zero_le n)
  have h :
      liftTransformer reflectionTransformer p5Family 0 n <
```

```
        operatorStage reflectionTransformer index p5Family n n :=
    Nat.lt_trans h0 h1
  change stage p5Family index n <
    operatorStage reflectionTransformer index p5Family n n
  rw [← liftReflection_eq_stage p5Family 0 n]
  exact h

/-- **The required comparison:** Contender 6 is strictly below Contender 7. -/
theorem contender_6_lt_contender_7 : contender_6 < contender_7 :=
  finalFamily_lt_autonomousFamily 42

/-- Hence the exact Lean port of Contender 5 is also strictly below
Contender 7. -/
theorem contender_5_lt_contender_7 : contender_5 < contender_7 :=
  Nat.lt_trans contender_5_lt_contender_6 contender_6_lt_contender_7

/-- Proposal P5's number is strictly below Contender 7. -/
theorem p5_lt_contender_7 : BigNumber.contender_6 < contender_7 :=
  Nat.lt_trans p5_lt_contender_6 contender_6_lt_contender_7

/-- Every operator-stage generated by a visible defined tree is strictly below
the final autonomous family. -/
theorem defined_operator_lt_autonomousFamily {k i n : Nat}
    (hk : k ≤ n) (hi : i ≤ n) :
    operatorStage reflectionTransformer
        (defTree (nthD (BigNumber.allTerms k) i .zero)) p5Family 0 n <
      autonomousFamily n n := by
  have h1 :
      operatorStage reflectionTransformer
          (defTree (nthD (BigNumber.allTerms k) i .zero)) p5Family 0 n <
        operatorStage reflectionTransformer (treesOfDepth k) p5Family 0 n :=
    operatorStage_child_lt reflectionTransformer
      reflectionTransformer_progressive p5Family
      (fun j => defTree (nthD (BigNumber.allTerms k) j .zero))
      (i := i) (o := 0) (n := n) hi
  have h2 :
      operatorStage reflectionTransformer (treesOfDepth k) p5Family 0 n <
        operatorStage reflectionTransformer index p5Family n n :=
    operatorStage_child_lt reflectionTransformer
      reflectionTransformer_progressive p5Family treesOfDepth
      (i := k) (o := n) (n := n) hk
  exact Nat.lt_trans h1 h2

/-- Every tree defined by the W-language gives an operator-stage that is
(eventually, at its canonical output coordinate) dominated by Contender 7's
family. -/
theorem defined_operator_eventually_lt {k : Nat} {t : Term}
    (ht : t ∈ BigNumber.allTerms k) :
    ∃ N, ∀ n, N ≤ n →
      operatorStage reflectionTransformer (defTree t) p5Family 0 n <
        autonomousFamily n n := by
  obtain ⟨i, hi⟩ := exists_nthD_of_mem (d := Term.zero) ht
  refine ⟨k + i, fun n hn => ?_⟩
  rw [← hi]
  exact defined_operator_lt_autonomousFamily
    (Nat.le_trans (Nat.le_add_right k i) hn)
    (Nat.le_trans (Nat.le_add_left i k) hn)

end BusyLean
```

C Tests

C.1 Smoke test

```
import BusyLean.MetaReflection
```

```
open BusyLean
open BusyLean.W
open BigNumber (WTree)

/-- A tiny executable transformer used only for smoke testing the recursion. -/
def plusOneTransformer : Transformer :=
  fun seed _ n => seed 0 n + 1

private def seed : Family := fun i n => i + n
private def oneNode : WTree Nat := .node fun _ => .leaf

#eval runTransformer plusOneTransformer seed .leaf 0 2
#eval runTransformer plusOneTransformer seed oneNode 0 2

example : runTransformer plusOneTransformer seed .leaf 0 2 = 3 := rfl
example : runTransformer plusOneTransformer seed oneNode 0 2 = 4 := rfl
```

C.2 Axiom audit

```
import BusyLean.Comparison7
import Lean

/-!
Run with:

    lake env lean test/Contender7Audit.lean

The expected axiom set is empty or a subset of `{propext, Quot.sound}`.
The dependency audit checks the transitive kernel-environment closure, not just
surface syntax.  Do not add `#eval BusyLean.contender_7`.
-/

open Lean Elab Command

/-- All constants reachable from `root` through declaration types and values. -/
def contender7ConstClosure (env : Environment) (root : Name) : NameSet := Id.run do
  let mut seen : NameSet := {}
  let mut stack := #[root]
  while stack.size > 0 do
    let c := stack.back!
    stack := stack.pop
    if seen.contains c then continue
    seen := seen.insert c
    let some ci := env.find? c | continue
    let body : Array Name := match ci with
      | .defnInfo v => v.value.getUsedConstants
      | .thmInfo v => v.value.getUsedConstants
      | .opaqueInfo v => v.value.getUsedConstants
      | _ => #[]
    stack := stack ++ ci.type.getUsedConstants ++ body
  return seen

/-- Fail if any listed declaration occurs in the transitive closure of root. -/
elab "#assert_contender7_not_in_closure " root:ident " : " bad:ident* : command => do
  let env ← getEnv
  let r ← liftCoreM <| realizeGlobalConstNoOverloadWithInfo root
  let closure := contender7ConstClosure env r
  for b in bad do
    let n ← liftCoreM <| realizeGlobalConstNoOverloadWithInfo b
    if closure.contains n then
      throwError m!"{r} depends on forbidden constant {n}"
  logInfo m!"{r}: closure of {closure.size} constants avoids all {bad.size} forbidden names"

#print axioms BusyLean.contender_7
#print axioms BusyLean.contender_6_lt_contender_7
#print axioms BusyLean.finalFamily_lt_autonomousFamily
#print axioms BusyLean.defined_operator_eventually_lt
```

```
#assert_contender7_not_in_closure BusyLean.contender_7 :
  BusyLean.contender_6 BusyLean.finalFamily BusyLean.contender_5
  BigNumber.contender_6 BigNumber.contender_5
  BusyLean.C5.largestOfDepth BusyLean.C5.maxBy
  BusyLean.C5.largestSTLCNatRecNatOfDepth

#check @BusyLean.contender_6_lt_contender_7
#check @BusyLean.contender_5_lt_contender_7
#check @BusyLean.p5_lt_contender_7
```

References

[1] V. Reshetnikov, *BusyLean: name the biggest number, in Lean*, repository revision `86b360648163f0249eab79539438b6a83dfe8215`, October 2026.

[2] C. Roux, *name-the-biggest-number*, original Coq contest repository, with Contenders 4 and 5.

[3] S. Aaronson, *Who Can Name the Bigger Number?*, expository essay on large-number contests and definability.

[4] J.-Y. Girard, Y. Lafont, and P. Taylor, *Proofs and Types*, Cambridge Tracts in Theoretical Computer Science, 1989.

[5] W. Pohlers, *Proof Theory: The First Step into Impredicativity*, Springer, 2009.